

Méthode de Travail du Projet

MSP2 — Equipe de Développement & Produit · Juillet 2026

VISION PÉDAGOGIQUE

Le Projet comme Accélérateur de Compétences

Au-delà du produit final (MVP), ce projet a été conçu avant tout comme un terrain d'apprentissage d'excellence pour développer vos compétences sur des aspects complexes et modernes du développement logiciel. L'objectif est de vous faire passer un cap technique à travers trois piliers :



Maîtrise d'une stack et d'une architecture modernes

Vous allez travailler sur le découpage en couches (UI, domaine, services, data) appliqué à un monolithe Next.js robuste. C'est ce niveau d'exigence architecturale qui sépare les développeurs juniors des profils seniors recherchés sur le marché.



Culture de la granularité

Apprendre à concevoir du code hautement découplé, où chaque feature est autonome, possède ses propres contrats de données et sa propre suite de tests. Maîtriser cette complexité logicielle vous permettra d'aborder n'importe quel grand projet d'entreprise avec assurance.



Pionniers du développement agentique

Le code ultra-granulaire et standardisé imposé ici n'est pas qu'un exercice de style : c'est l'architecture idéale pour travailler main dans la main avec des agents IA. En apprenant à structurer votre code pour qu'il soit facilement lisible et modifiable par une IA, vous développez une compétence rare : le rôle de « développeur-pilote », capable de guider des agents pour produire du code de qualité à haute vitesse sans créer de dette technique.

Sommaire

1 Déroulé de travail

2 Parties du projet

3 Choses à savoir

1. Déroulé de travail

1.1 Conception pure, puis développement

Conception pure (par module)

Chaque propriétaire de partie produit un pack de conception autonome, sans écrire le code métier de production.

- Maquettes UI/UX (écrans, flux, états vides / chargement / erreur)
- Diagrammes utiles (séquences, entités, flux de données, architecture cible du module)
- Choix technologiques locaux, explicitement compatibles avec Next.js et les conventions du projet
- Liste des dépendances, risques et points d'intégration avec les autres modules

Validation de conception (porte d'entrée)

Aucune bascule vers le développement métier sans feu vert sur la conception du module concerné.

- Revue de cohérence globale (vision produit, SEO/GEO, sécurité, RGPD)
- Vérification de compatibilité technique avec le socle Next.js prévu
- Clarification des interfaces entre modules (contrats API, formats de données, responsabilités)
- Ajustements demandés puis revalidation si nécessaire

Socle Next.js (directeur de projet)

Le directeur de projet crée la structure principale de l'application : ossature commune que toute l'équipe devra suivre.

- Initialisation Next.js (App Router, conventions de dossiers, linting, scripts)
- Socle transverse : layout, routing de base, config env, styles / design tokens partagés
- Emplacements réservés pour brancher les modules (features) sans casser le monolithe MVP
- Documentation courte : « comment ajouter un block » et conventions de nommage

Développement des blocks (contributeurs)

Chaque groupe reprend le socle validé et y intègre son module en respectant l'architecture imposée.

- Implémentation alignée sur la conception validée (pas de réinvention ad hoc)
- Branches / PR ciblées sur un block, relecture et démo de sprint
- Adaptation aux conventions du dépôt (dossiers, API Routes, typage, tests)
- Signalement rapide de tout écart de conception découvert en cours de code

1.2 Responsabilités & livrables de conception

Règle d'or : **qui possède un module en possède aussi la conception**. Le propriétaire (personne ou groupe) livre un pack complet, prêt à être relu, avant toute écriture de code métier.

- Propriétaire de module — conception totale de sa partie (maquettes, diagrammes, stack locale compatible, critères d'acceptation).
- Directeur de projet — arbitrage global, validation des conceptions, création et maintenance du socle Next.js.
- Contributeurs — reprise du socle, intégration du block, respect des conventions et des interfaces validées.
- Leaders techniques de module — mentorat sur leur expertise (UI, IA, données, tests) pendant conception puis développement.

Le pack de conception minimal attendu par module : objectif du module, parcours utilisateur, maquettes des écrans clés, diagramme d'intégration, choix techno justifiés, dépendances / risques, et définition de « done » pour la validation.

1.3 Principes à respecter

- Conception d'abord : pas de développement métier sans conception validée du module.
- Compatibilité Next.js : toute techno proposée doit s'intégrer au monolithe MVP (SSR/SEO, API Routes, portabilité future).
- Socle unique : on ne multiplie pas les bootstraps — chacun part de la structure créée par le directeur de projet.
- Contrats entre modules : les interfaces (API, données, UI partagée) sont figées à la validation, puis éventuellement ajustées en revue.
- Cascade pour le cadre, Agile pour l'exécution : la trajectoire (conception + socle) est cadrée ; les sprints portent l'implémentation des blocks.

2. Parties du projet

Choisir un module

Chaque module est présenté comme un niveau : lisez le profil (Logique, Créatif), les contraintes et les points d'attention, puis choisissez la partie qui vous convient.

Logique

Complexité algorithmique, data, règles métier.

Créatif

UI/UX, pédagogie, direction artistique.

FEAT-01

CRÉATEUR

Création de Contenu

Feature primordiale : le module de création de contenu dédié au site. Il permet aux utilisateurs de créer différents types de contenus (cours, quiz, etc.), personnalisables selon leurs besoins et leur environnement, puis de partager leurs créations pour une apparition dans le feed.

Point d'attention majeur : ces créations sont l'essence même de la communication réalisée sur la plateforme. Les cours, par exemple, seront rendus sous forme de pages statiques devant respecter au maximum les règles SEO — le référencement du site s'appuiera ainsi sur du contenu édité par les utilisateurs.

L'équipe en charge conceptualise l'ensemble du module : de la structure des contenus jusqu'à leur utilisation (création, personnalisation, publication, partage, rendu).

LOGIQUE



CRÉATIF



CONTRAINTES

- Multi-types de contenus : cours, quiz, et extensions possibles selon les besoins.
- Personnalisation forte selon le contexte / environnement de chaque créateur.
- Partage vers le feed (FEAT-02) après publication.
- Rendu pages statiques SSR / SEO maximal pour les cours (référencement via contenu user).
- Format de sauvegarde léger, lisible et éditable (y compris par des modèles de langage).

POINTS D'ATTENTION

- Conceptualiser tout le périmètre : structure des contenus ' création ' usage ' publication / SEO.
- Ne pas dériver vers un traitement de texte généraliste : borner le MVP aux besoins de contenus structurés (cours, quiz...).

Flux d'Actualité C2C

Cette feature consiste à développer l'interface principale des utilisateurs — le point d'entrée quotidien de la plateforme.

Concrètement, il s'agit de construire un feed moderne en swipe vers le haut, en s'inspirant des usages des réseaux sociaux actuels, tout en l'adaptant au contexte d'apprentissage. Le fil doit être infini et accepter divers types de médias : vidéos, posts, photos, et d'autres formats selon les besoins.

L'expérience utilisateur doit rester fluide : une attention particulière sera portée au cache côté front (préchargement, conservation des éléments déjà vus, limitation des rechargements inutiles) afin d'éviter les saccades et les attentes inutiles.

L'équipe en charge du feed développe aussi l'algorithme d'affichage. Pour des raisons de performance, l'approche conseillée est la suivante : les données utilisateur pertinentes sont envoyées vers le front ; l'algo côté client effectue sa recette (scoring / filtrage) pour décider quoi demander ensuite au back. On évite ainsi de tout calculer côté serveur à chaque scroll.

Cette responsabilité revient à l'équipe du feed car les données utiles au ranking ne viendront pas seulement du fil lui-même : elles seront aussi collectées via les autres parties du projet. Dans un premier temps, les affichages s'appuient sur des jeux de données de test couvrant les besoins du feed ; la collecte réelle sera branchée ensuite (FEAT-02-b).

LOGIQUE



CRÉATIF



CONTRAINTES

- Feed infini multi-médias (vidéo, post, photo, etc.).
- Cache front soigné pour une expérience fluide (swipe, préchargement, moins de refetch).
- Algo d'affichage côté front : données user ' recette client ' requêtes ciblées au back.
- Jeux de test d'abord ; collecte réelle en FEAT-02-b.
- Définir dès FEAT-02 le contrat / type de données attendu (02-b ne fait que la récupération).

POINTS D'ATTENTION

- L'algo doit pouvoir consommer des signaux issus du feed et des autres modules.
- Ne pas sacrifier la perf UX : mesurer tôt le coût du cache et du préchargement média.

Collecte de Données Utilisateur

Suite de FEAT-02 : une fois le feed en place avec des jeux de test, cette partie vise à définir et développer les algorithmes (côté client / pipeline associé) qui récupèrent les données importantes pour alimenter le ranking et l'affichage.

Le type de données attendu devra déjà être construit et figé en FEAT-02 (contrats, formes, signaux nécessaires au feed). Ici, on ne redéfinit pas le modèle : on se concentre uniquement sur la récupération de ces données.

L'objectif est de collecter le maximum de signaux utiles côté client, tout en respectant le RGPD et en optimisant au maximum le travail front : pas de surcharge inutile, pas de payloads massifs à chaque interaction.

Une piste concrète : récupérer les données brutes de comportement, puis appliquer des algorithmes de scoring qui n'envoient au front qu'un résumé compact (profil de préférences, scores agrégés). Le feed consomme ainsi des signaux déjà digérés, sans retravailler toute l'historique à chaque scroll.

Ces algos cherchent surtout à exploiter des données qui n'ont pas vocation à entrer en base telles quelles. Exemple : mesurer le temps passé sur chaque vidéo ou post, croiser ces signaux avec d'autres interactions, puis en déduire quel type de contenu l'utilisateur préfère — sans stocker le détail brut de chaque seconde de visionnage.

LOGIQUE



CRÉATIF



CONTRAINTES

- Développé après FEAT-02 : le contrat / type de données est déjà défini dans le feed ; ici = récupération uniquement.
- Collecte maximale de signaux clients, compatible RGPD / consentement (FEAT-07).
- Scoring ' résumés compacts envoyés au front ; éviter les gros transferts de données brutes.
- Privilégier des données comportementales éphémères ou agrégées, pas un dumping brut en base.

POINTS D'ATTENTION

- Exemple cible : temps passé par média + croisement ' préférence de type de contenu.
- Documenter clairement ce qui est collecté, agrégé, résumé, et ce qui n'est jamais persisté.

FEAT-03

SYSTÈME

Système de Gamification Avancé

Cette partie est spécifique : elle ne s'intégrera pas immédiatement au cœur du projet. L'enjeu est d'abord d'imaginer et de construire les données, règles et mécaniques que l'on pourra conserver et brancher plus tard pour donner un vrai côté ludique à la plateforme.

Exemples de pistes : un niveau d'expérience lié aux quiz, qui fait progresser le niveau des utilisateurs ; ou encore des rôles / guildes pour celles et ceux qui souhaitent publier en groupe. L'idée est d'emprunter au RPG — progression visible, rôles, éventuellement récompenses — pour rendre l'usage plus engageant, voire un peu addictif.

L'équipe en charge devra donc comprendre le fonctionnement général de la plateforme (parcours, modules, interactions) afin d'y intégrer un sens ludique cohérent : pas un gadget isolé, mais une couche de progression qui s'appuie sur ce que font déjà les utilisateurs.

LOGIQUE



CRÉATIF



CONTRAINTES

- Conception / modèle de données d'abord ; intégration runtime différée, pas bloquante pour le MVP immédiat.
- Mécaniques type RPG : XP, niveaux, rôles / guildes, récompenses — progression visible.
- Conçu pour s'intégrer plus tard : API / hooks clairs vers quiz, posts, groupes et autres modules.
- Règles de progression déterministes et testables (pas uniquement côté client).

POINTS D'ATTENTION

- Bien cartographier la plateforme avant de figer les mécaniques ludiques.
- Équilibrer engagement / côté addictif et objectif pédagogique — éviter le « farm » vide de sens.

FEAT-04

SYSTÈME

Service IA & Tokenisation

Dans un premier temps, il s'agit de développer un service de communication avec des modèles d'IA, conçu pour être réutilisable par les autres modules du projet selon leurs besoins.

Le service inclut la tokenisation (suivi / consommation de tokens) et doit s'adapter à plusieurs sources externes : Gemini (gratuit, adapté au développement) et Ollama (visé pour la production).

L'objectif est un fonctionnement plug and play : changer de provider ou de modèle ne doit demander que la modification des paramètres fournisseur dans les variables d'environnement — sans réécrire les modules consommateurs.

LOGIQUE



CRÉATIF



CONTRAINTES

- Service de communication IA réutilisable par les autres features.
- Tokenisation intégrée (quota / consommation traçable).
- Multi-providers : Gemini (dev) et Ollama (prod) a minima.
- Plug and play via env : changement de provider / modèle sans toucher au code métier.
- Appels via API Routes serveur ; clés jamais exposées au client.

POINTS D'ATTENTION

- Prévoir les problèmes liés aux modèles et providers externes (pas de réponse, réponse incohérente, timeout, erreur d'API...).
- RGPD : ne pas envoyer de données personnelles inutiles au provider.
- Suite prévue en FEAT-04-b : intégration métier du service dans les autres features.

FEAT-04-b

SYSTÈME

Intégration IA Métier

Dans un second temps, une fois le service de communication IA (FEAT-04) en place, l'objectif est de permettre son intégration aux autres features du projet.

Il s'agit d'adapter les prompts et le comportement du service à nos usages métier, pour créer des fonctions IA concrètes destinées aux utilisateurs — et non plus seulement une brique technique générique.

Exemple : une fonctionnalité permettant à l'utilisateur de développer / enrichir son cours via les modèles (suggestions de structure, reformulation, génération d'étapes, etc.), en s'appuyant sur le service plug and play déjà construit.

LOGIQUE



CRÉATIF



CONTRAINTES

- Dépend de FEAT-04 : le service de base (providers, tokenisation, env) doit déjà fonctionner.
- Prompts et orchestration adaptés aux cas métier (cours, feed, etc.), pas d'appels IA « bruts » côté UI.
- Fonctions IA réutilisables / appelables depuis les autres modules sans dupliquer le client IA.
- Respect des quotas tokens et du RGPD hérités de FEAT-04.

POINTS D'ATTENTION

- Chaque use case métier doit avoir un contrat clair (entrées, sorties, limites) avant d'être exposé aux users.
- Éviter de coupler trop fort une feature métier à un provider précis : rester sur l'abstraction FEAT-04.

FEAT-06**SYSTÈME****Gestion de Données de Masse**

Cette feature vise la gestion de données nombreuses et temps réel : messageries, commentaires, et salons (espaces de groupe créés par des référents).

L'enjeu principal n'est pas seulement l'UI, mais une logique performante de transmission en direct des messages et contenus, capable d'absorber un volume important d'interactions.

Dans un premier temps, pour faciliter le développement, le service pourra être interne au projet. Il ne devra toutefois pas en dépendre fortement : en production réelle, ce service a vocation à être externalisé (hébergement / infra dédiée) sans réécriture majeure.

LOGIQUE**CRÉATIF****CONTRAINTES**

- Couvrir messagerie, commentaires et salons (groupes créés par référents).
- Logique performante pour la transmission en direct (messages / contenus).
- Implémentation interne possible au MVP, mais architecture découplée / externalisable en prod.
- Éviter un couplage fort au monolithe Next.js (interfaces claires, remplacement du backend de flux possible).

POINTS D'ATTENTION

- Concevoir dès le départ les contrats (API / événements) comme si le service était déjà externe.
- Mesurer tôt latence et charge : pagination, curseurs, rate limiting si besoin.

FEAT-07**SYSTÈME****Données Utilisateur, Consentements & Cookies**

Créer un service de gestion des données et paramètres utilisateur, ainsi que la gestion des cookies. Ce module est le point central de la conformité et du contrôle utilisateur sur tout ce qui est collecté sur la plateforme.

Le travail commence par un examen des différents services de collecte du site (feed, analytics, cookies, etc.) : quelles données sont prises, dans quel but, et quels sont les droits légaux associés selon le type — données de fonctionnement, données analytics, données personnelles, etc. L'utilisateur dispose de droits sur ces données ; il faut les cartographier clairement.

L'équipe analyse chaque service de collecte et rattache chaque donnée à un rôle / finalité (nécessaire au fonctionnement, analytics optionnel, personnalisation, etc.). L'utilisateur pourra refuser le consentement à l'analyse de certaines données : le système doit alors permettre de récupérer facilement la liste des droits / consentements encore accordés, pour que les services de collecte fonctionnent uniquement dans le périmètre autorisé.

En plus de cette couche de droits et de consentement, l'équipe est aussi en charge du panneau de gestion des données utilisateur (paramètres, cookies, export / suppression, choix de consentement).

LOGIQUE



CRÉATIF



CONTRAINTES

- Service unique : paramètres utilisateur + cookies + registre des consentements.
- Cartographie légale par type de donnée (fonctionnement, analytics, personnelles...) et par service collecteur.
- Chaque donnée rattachée à un rôle / finalité ; refus de consentement ' les collecteurs ne voient que le périmètre encore autorisé.
- API simple pour interroger « quels droits avons-nous pour ce service / cet utilisateur ? ».
- Panneau utilisateur : gestion données, cookies, consentements, export / suppression.

POINTS D'ATTENTION

- Distinguer clairement données strictement nécessaires au fonctionnement et données soumises à consentement.
- Impact transverse : tous les modules de collecte doivent consommer ce service avant de collecter.

FEAT-08

SYSTÈME

SEO, Sitemap & Protection du Contenu

Gérer l'exportation SEO via des modules réutilisables par les autres features (métadonnées, balises, JSON-LD, etc.). L'objectif est que chaque module métier puisse brancher son contenu sur une couche SEO commune, sans réinventer le référencement à chaque fois.

Parmi les responsabilités concrètes : le service de sitemap dynamique, qui guide correctement les robots pour l'indexation des pages de la plateforme.

Particularité importante : la protection intellectuelle des données du site. Il faut publier assez d'information pour un bon référencement, sans tout dévoiler gratuitement et facilement. L'équipe doit donc concevoir une solution de référencement à divulgation partielle : les bots voient ce qu'il faut pour indexer, pas la totalité du contenu utile.

Exemple : si un utilisateur crée un cours, les robots peuvent accéder à une page / aperçu indexable pour le SEO, sans exposer l'intégralité du cours (contenu réservé, paywall léger, extrait, rendu différencié bot vs utilisateur authentifié — selon la stratégie retenue).

LOGIQUE



CRÉATIF



CONTRAINTES

- Modules SEO réutilisables par les autres features (pas de duplication métier).
- Sitemap dynamique pour une indexation correcte par les robots.
- Référencement avec divulgation partielle : assez pour le SEO, pas le contenu complet en clair.
- Cours / contenus user : page bot / aperçu indexable ` accès total au contenu.
- SSR, balises et JSON-LD cohérents avec Next.js / App Router.

POINTS D'ATTENTION

- Équilibre fragile entre SEO (visibilité) et protection IP (ne pas offrir le contenu entier aux scrapers).
- Documenter clairement ce que voient les bots vs les utilisateurs connectés / ayants droit.

3. Choses à savoir

Repères pour l'équipe

Points essentiels à garder en tête : maquettes, thèmes, et les mots-clés Granularité & Réutilisabilité pour une architecture pro et un développement agentique efficace.

Maquettes : aucune charte graphique imposée

Pour les maquettes, aucune charte graphique n'est imposée. Vous êtes libres de choisir le style qui vous convient. Le projet s'appuiera sur une structure React + Tailwind : si possible, privilégiez des maquettes HTML compatibles pour faciliter le développement.

Thèmes : votre charte deviendra celle du site

Le projet sera ensuite doté d'un dossier de thèmes. Vous pourrez y créer votre propre thème à partir de votre maquette. Tout le site prendra alors les couleurs de la charte graphique que vous aurez définie. En conséquence, le développement de tous les composants UI devra respecter le cadre de valeurs du thème (couleurs, espacements, typographie, etc.) — pas de styles « en dur » hors de ce système.

Mots-clés : Granularité & Réutilisabilité

Deux mots-clés à retenir pour tout le développement : Granularité et Réutilisabilité. Ils poussent à découper le code en couches nettes (UI, domaine, services, data...) et en modules petits, ciblés, facilement réutilisables. Ce n'est pas seulement de « la belle architecture » : c'est aussi le meilleur terrain pour progresser en développement agentique. Un code granulaire se laisse lire et modifier par un agent IA sans confusion ; un code réutilisable limite les doublons et les régressions. En travaillant ainsi, vous améliorez concrètement vos skills : mieux cadrer une tâche pour un agent, obtenir des changements plus propres, et monter en compétence à la fois en architecture et en collaboration homme-agent.

L'architecture du projet sera définie à l'avance pour guider le développement en couches granulaires.

Limiter les librairies externes

On évite au maximum les librairies externes pour la logique métier. Le principe : concevoir et développer nous-mêmes ce qui est spécifique à notre usage. Avec le développement agentique, on peut produire beaucoup de code ciblé en peu de temps — assez pour couvrir des features entières, taillées pour le projet, sans dépendre d'un package générique.

Avantages : moins d'imports de petites libs potentiellement vulnérables ; pas de dépendance lourde pour un usage partiel ; code 100 % modulable selon nos besoins ; moins de dette externe pour la maintenabilité. Depuis l'essor du développement assisté par IA, ces bénéfices l'emportent largement sur le gain de temps d'une lib déjà prête — dès lors qu'il s'agit de logique métier « maison ».

Exception : pour les fonctionnalités complexes ou à enjeu sécurité (auth, crypto, parsing critique, etc.), on s'appuie sur des librairies éprouvées plutôt que de réinventer la roue.

Tests : intégrés au socle, portés par chaque feature

La feature « tests » a été retirée du catalogue : l'architecture de test arrive avec le code de base (socle Next.js). Le harnais est prêt dès le démarrage.

Les tests reposent sur Playwright, dans une structure adaptée au monolithe. Chaque feature écrit et maintient ses propres tests (parcours UI, non-régression), selon les conventions du socle commun.